

Week 6 - Advanced Security Audits & Final Deployment Security

1.0 Overview

The objective of Week 6 was to conduct advanced security audits, ensure compliance with industry standards, and prepare the application for secure deployment. The three main areas of focus were Security Audits & Compliance (OWASP ZAP, Nikto, Lynis), Secure Deployment Practices (dependency scanning, Docker best practices) and Final Penetration Testing & Documentation

2.0 Security Audits & Compliance

2.1 OWASP ZAP Automated Scan

2.1.1 What is OWASP ZAP?

OWASP ZAP (Zed Attack Proxy) is a free, open-source web application security scanner maintained by OWASP. It is used by both developers and penetration testers to find security vulnerabilities in web applications. ZAP can perform both passive scanning (observing traffic) and active scanning (actively probing for vulnerabilities).

2.1.2 Running the Scan

ZAP was launched and an automated scan was performed against the Juice Shop application:

Target: `http://localhost:3000`

ZAP Version: 2.17.0

Scan type: Automated (Spider + Active Scan)

Nodes scanned: 163

2.1.4 Notable Findings

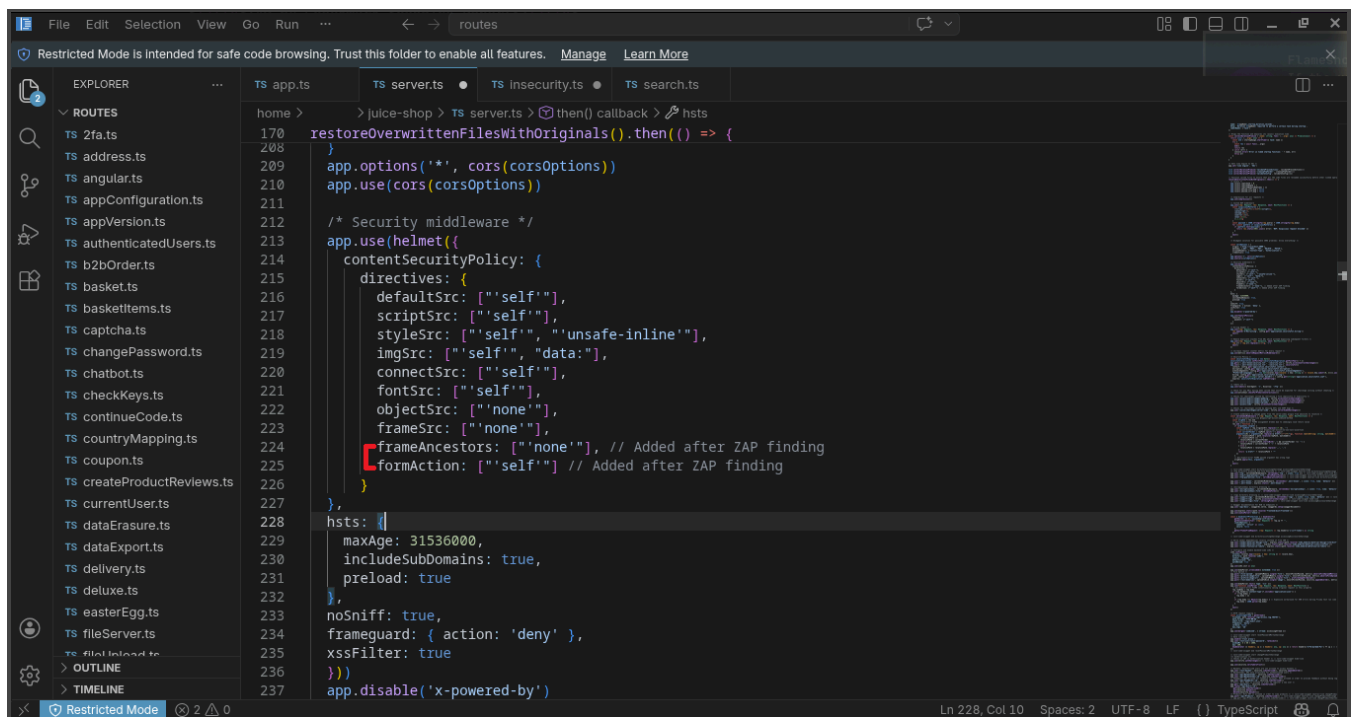
The most significant finding from the ZAP scan was that zero high-severity vulnerabilities were detected. This confirms that the security hardening applied in Weeks 4 and 5 was effective (discussed further below):

The security scan produced positive results, with the SQL injection test returning zero alerts, confirming that the Week 5 parameterized query fix was successfully implemented. The XSS scan also returned zero alerts, indicating that the Content Security Policy (CSP) is functioning effectively. Additionally, no path traversal, command injection, buffer overflow, or server-side request forgery (SSRF) vulnerabilities were detected during the assessment, suggesting that these common attack vectors are adequately mitigated.

Most medium alerts were related to Socket.IO WebSocket endpoints, which bypass Express middleware and therefore do not inherit the security headers configured via helmet.js.

2.1.5 CSP Fix Applied

To address the missing frame-ancestors and form-action directives identified by ZAP, the helmet CSP configuration in server.ts was updated:



```
File Edit Selection View Go Run ... routes
Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. Manage Learn More

EXPLORER
TS 2fa.ts
TS address.ts
TS angular.ts
TS appConfiguration.ts
TS appVersion.ts
TS authenticatedUsers.ts
TS b2bOrder.ts
TS basket.ts
TS basketItems.ts
TS captcha.ts
TS changePassword.ts
TS chatbot.ts
TS checkKeys.ts
TS continueCode.ts
TS countryMapping.ts
TS coupon.ts
TS createProductReviews.ts
TS currentUser.ts
TS dataErasure.ts
TS dataExport.ts
TS delivery.ts
TS deluxe.ts
TS easterEgg.ts
TS fileServer.ts
TS fileIntend.ts
> OUTLINE
> TIMELINE

TS app.ts TS server.ts TS insecurity.ts TS search.ts
home > > juice-shop > TS server.ts > then() callback > hsts
170 restoreOverwrittenFilesWithOriginals().then(() => {
208 }
209 app.options('*', cors(corsOptions))
210 app.use(cors(corsOptions))
211
212 /* Security middleware */
213 app.use(helmet({
214   contentSecurityPolicy: {
215     directives: {
216       defaultSrc: ['self'],
217       scriptSrc: ['self'],
218       styleSrc: ['self', 'unsafe-inline'],
219       imgSrc: ['self', 'data:'],
220       connectSrc: ['self'],
221       fontSrc: ['self'],
222       objectSrc: ['none'],
223       frameSrc: ['none'],
224       frameAncestors: ['none'], // Added after ZAP finding
225       formAction: ['self'] // Added after ZAP finding
226     }
227   },
228   hsts: {
229     maxAge: 31536000,
230     includeSubDomains: true,
231     preload: true
232   },
233   noSniff: true,
234   frameguard: { action: 'deny' },
235   xssFilter: true
236 })))
237 app.disable('x-powered-by')
```

(Figure 2.1.3)

```

contentSecurityPolicy: {
  directives: {
    defaultSrc: ['"self"'],
    scriptSrc: ['"self"'],
    styleSrc: ['"self"', '"unsafe-inline"'],
    imgSrc: ['"self"', "data:"],
    connectSrc: ['"self"'],
    fontSrc: ['"self"'],
    objectSrc: ['"none"'],
    frameSrc: ['"none"'],
    frameAncestors: ['"none"'], // Added ZAP finding
    formAction: ['"self"'] // Added based on ZAP finding
  }
}

```

2.2 Nikto Web Scan

2.2.1 What is Nikto?

Nikto is an open-source web server scanner that performs comprehensive tests against web servers. It checks for dangerous files and programs, outdated server software, and various HTTP server configuration issues.

2.2.2 Running the Scan

```
nikto -h http://localhost:3000 2>&1 | tee recon_nikto.txt
```

```

+ [002748] /.mysql_history: Database SQL?.
+ [002756] /.sh_history: A user's home directory may be set to the web root
, the shell history was retrieved. This should not be accessible via the we
b.
+ ERROR: *** Error limit (20) reached for host, giving up. Last error: open
ing stream: can't connect (timeout): Transport endpoint is not connected. *
**
+ ERROR: *** Consider using mitmproxy to avoid TLS fingerprinting. ***
- STATUS: Completed 3920 requests (~54% complete, ~6.6 minutes left): curre
ntly in plugin 'Nikto Tests'
- STATUS: Running average: Not enough data.
+ Scan terminated: 20 errors and 10 items reported on the remote host
+ End Time:          2026-06-04 16:23:22 (GMT5) (465 seconds)
-----
+ 1 host(s) tested
[archlinux juice-shop]$

```

(Figure 2.2.1 - Nikto scan output showing findings and early termination message)

Key findings:

The assessment identified several security findings. Medium-severity issues included an exposed `/ftp/` directory with directory listing enabled, allowing files to be browsed, as well as public access to the `/.htpasswd`, `/.bash_history`, and `/.mysql_history` files, which could expose authorization information, shell command history, and database query history. A low-severity finding was the absence of the *Permissions-Policy* header, indicating that browser feature permissions were not fully configured. Informational findings included the disclosure of the `/ftp` directory through the `robots.txt` file and the presence of an uncommon `x-recruiting` HTTP header. These findings primarily relate to information disclosure and security configuration weaknesses.

Note: The Nikto scan was terminated early after generating 20 connection errors. These errors were caused by the Week 4 rate limiter returning 429 responses, which caused Nikto's connections to time out. This is a positive finding - it confirms that the rate limiting is actively defending against automated scanning tools.

2.3 Lynis System Audit

2.3.1 What is Lynis?

Lynis is an open-source security auditing tool for Unix-based systems. It performs an in-depth security scan of the system and provides a hardening index score along with specific suggestions for improvement. It checks areas including boot security, user authentication, SSH configuration, kernel hardening, file permissions, and installed software.

2.3.2 Running the Audit

```
sudo lynis audit system 2>&1 | tee ~/juice-shop/lynis_report.txt
```

```

=====
-[ Lynis 3.1.6 Results ]-

Great, no warnings

Suggestions (42):
-----
* Set a password on GRUB boot loader to prevent altering boot configuration (e.g. boot in single user mode without password) [BOOT-5122]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/BOOT-5122/

* Consider hardening system services [BOOT-5264]
  - Details : Run '/usr/bin/systemd-analyze security SERVICE' for each service
  - Related resources
    * Article: Systemd features to secure service files: https://linux-audit.com/systemd/systemd-features-to-secure-units-and-services/
    * Website: https://cisofy.com/lynis/controls/BOOT-5264/

* If not required, consider explicit disabling of core dump in /etc/security/limits.conf file [KRNL-5820]
  - Related resources
    * Article: Understand and configure core dumps on Linux: https://linux-audit.com/software/understand-and-configure-core-dumps-work-on-linux/
    * Website: https://cisofy.com/lynis/controls/KRNL-5820/

* Configure password hashing rounds in /etc/login.defs [AUTH-9230]
  - Related resources
    * Article: Linux password security: hashing rounds: https://linux-audit.com/authentication/configure-the-minimum-password-length-on-linux-systems/
    * Website: https://cisofy.com/lynis/controls/AUTH-9230/

* When possible set expire dates for all password protected accounts [AUTH-9282]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/AUTH-9282/

```

(Figure 2.3.1 - Lynis listing the 42 recommendations + No warnings)

```

- Security audit      [V]
- Vulnerability scan  [V]

Details:
Hardening index : 67 [#####          ]
Tests performed : 244
Plugins enabled : 0

Software components:
- Firewall            [X]
- Intrusion software  [V]
- Malware scanner     [V]

Files:
- Test and debug information : /var/log/lynis.log
- Report data                : /var/log/lynis-report.dat

=====

Notice: This version of Lynis is older than 6 months and might be outdated. Check the project page if a newer version is available.

=====

Lynis 3.1.6

Auditing, system hardening, and compliance for UNIX-based systems
(Linux, macOS, BSD, and others)

2007-2025, CISOfy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)

=====

[TIP]: Enhance Lynis audits by adding your settings to custom.prf (see /etc/lynis/default.prf for all settings)

archlinux ~]$

```

(Figure 2.3.2 - Lynis results summary showing hardening index of 67/100)

2.3.3 Results

Hardening index : 67 [#####]
Tests performed : 244
Warnings : 0
Suggestions : 42

Key findings by category:

The system audit identified several security weaknesses across multiple categories. The highest-priority finding was the absence of an active firewall, which increases the system's exposure to unauthorized network access. Medium-priority issues included SSH configuration weaknesses such as *AllowTcpForwarding* being enabled and *MaxAuthTries* being set to 6 instead of the recommended value of 3. Additional medium-priority findings were insufficient kernel hardening due to multiple non-secure *sysctl* settings, outdated system packages, the lack of the *auditd* auditing service, and the absence of GRUB password protection. Lower-priority issues included SSH running on the default port 22 and compiler tools being accessible to non-root users. Together, these findings indicate several opportunities to strengthen the system's overall security posture through improved hardening and configuration management.

Positive findings:

The audit also identified several positive security controls that were already in place. Fail2Ban was installed and actively protecting against brute-force attacks, while a rootkit scanner (*rkhunter*) and malware scanning tools were available to help detect malicious activity. PAM password strength policies were configured to enforce stronger user passwords, and no zombie processes were detected on the system. Logging was enabled through the *systemd* journal service, providing valuable audit and troubleshooting information. Additionally, AppArmor was present, offering an extra layer of application-level access control and system protection. These measures contribute positively to the overall security posture of the system.

2.4 OWASP Top 10 Compliance Check

A01 - Broken Access Control:

Compliance with OWASP A01 was achieved through the enforcement of JWT-based authentication on all protected routes. This helps ensure that users can only access resources for which they are authorized.

A02 - Cryptographic Failures:

Compliance with OWASP A02 is partially achieved. While HTTPS is not enforced in the local development environment, the application is configured with HSTS headers for production deployments to ensure secure communications.

A03 - Injection:

Compliance with OWASP A03 was successfully achieved. The SQL injection vulnerability previously identified in the product search endpoint was remediated during Week 5 through the implementation of parameterized queries.

A04 - Insecure Design:

Compliance with OWASP A04 is only partial. Some intentionally vulnerable features remain present as part of the Juice Shop training environment, meaning certain insecure design elements still exist by design.

A05 - Security Misconfiguration:

Compliance with OWASP A05 was achieved through the implementation of security best practices, including the use of Helmet.js security headers and the removal of the *x-powered-by* header to reduce information disclosure.

A06 - Vulnerable and Outdated Components:

Compliance with OWASP A06 remains outstanding. The dependency audit identified 64 npm package vulnerabilities that require remediation to fully address risks associated with vulnerable third-party components.

A07 - Identification and Authentication Failures:

Compliance with OWASP A07 was achieved by implementing rate limiting on login requests, restricting users to 10 requests every 15 minutes and helping protect against brute-force attacks.

A08 - Software and Data Integrity Failures:

Compliance with OWASP A08 is partially achieved. Although general integrity controls are present, Subresource Integrity (SRI) has not been implemented for external resources, leaving room for improvement.

A09 - Security Logging and Monitoring Failures:

Compliance with OWASP A09 was achieved through the use of Morgan access logging and an active Fail2Ban configuration, providing effective logging, monitoring, and automated response capabilities.

A10 - Server-Side Request Forgery (SSRF):

Compliance with OWASP A10 was achieved. Security testing with OWASP ZAP did not identify any SSRF vulnerabilities within the application.

3.0 Secure Deployment Practices

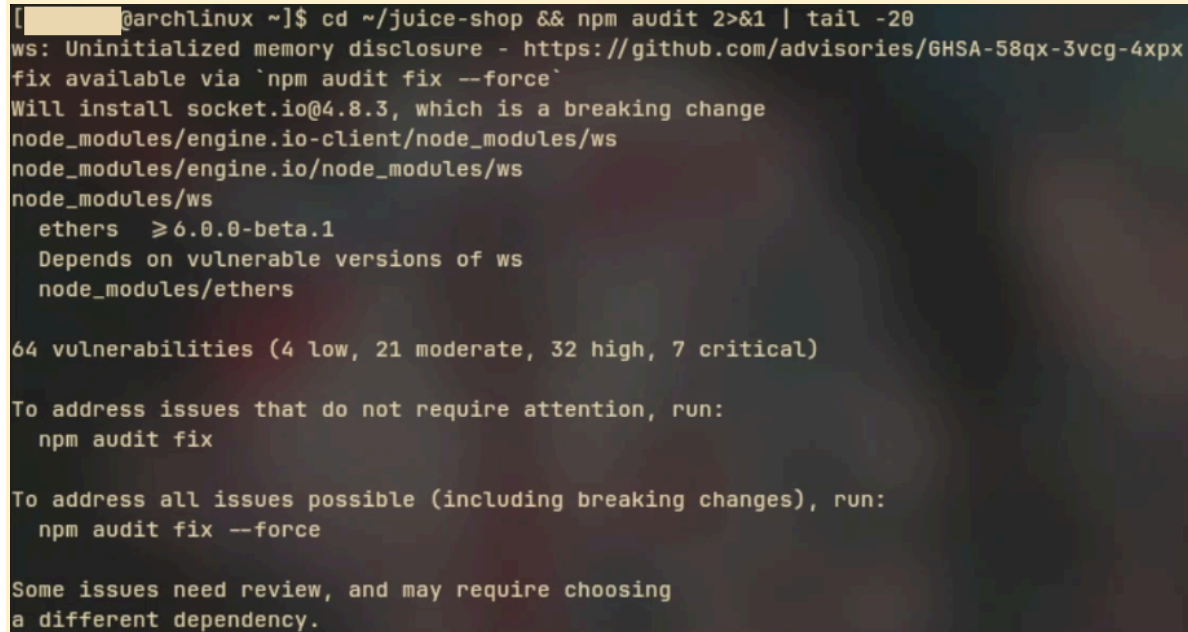
3.1 Dependency Vulnerability Scanning

3.1.1 What is Dependency Scanning?

Modern web applications rely on dozens or hundreds of third-party packages. Dependency scanning identifies known vulnerabilities in these packages by comparing them against public vulnerability databases such as the National Vulnerability Database (NVD).

3.1.2 Running the Scan

```
cd ~/juice-shop && npm audit 2>&1 | tail -20
```



```
[archlinux ~]$ cd ~/juice-shop && npm audit 2>&1 | tail -20
ws: Uninitialized memory disclosure - https://github.com/advisories/GHSA-58qx-3vcg-4xpx
fix available via `npm audit fix --force`
Will install socket.io@4.8.3, which is a breaking change
node_modules/engine.io-client/node_modules/ws
node_modules/engine.io/node_modules/ws
node_modules/ws
  ethers ≥6.0.0-beta.1
  Depends on vulnerable versions of ws
  node_modules/ethers

64 vulnerabilities (4 low, 21 moderate, 32 high, 7 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues possible (including breaking changes), run:
  npm audit fix --force

Some issues need review, and may require choosing
a different dependency.
```

(Figure 3.1.1 - 64 vulnerabilities along with other details)

Results:

64 vulnerabilities (4 low, 21 moderate, 32 high, 7 critical)

Notable vulnerability:

The dependency audit identified a notable vulnerability in the ws package, a WebSocket library used by Socket.IO and Ethers. The vulnerability, tracked under advisory *GHSA-58qx-3vcg-4xpx*, could lead to uninitialized memory disclosure under certain conditions. The recommended remediation is to run `npm audit fix --force`, which upgrades the affected dependency to a secure version. However, applying this fix requires a major-version upgrade to *socket.io@4.8.3*, introducing a potential breaking change that should be tested before deployment.

3.2 Docker Security Best Practices

Although the application was deployed from source rather than Docker for this assessment, the following Docker security best practices are documented for production deployment:

3.2.1 Secure Dockerfile

Use minimal base image to reduce attack surface

FROM node:20-alpine

Run as non-root user

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

USER appuser

Copy only production dependencies

COPY --chown=appuser:appgroup package*.json ./

RUN npm ci --only=production

Copy application files

COPY --chown=appuser:appgroup . .

EXPOSE 3000

CMD ["node", "build/app"]

3.2.2 Container Image Scanning

Scan container image for vulnerabilities using trivy

trivy image juice-shop:latest

Or using Docker Scout

docker scout cves juice-shop:latest

3.2.3 Key Docker Security Rules

- ★ Never run containers as root
- ★ Use read-only file systems where possible: *--read-only*
- ★ Drop all Linux capabilities: *--cap-drop=ALL*
- ★ Limit container resources: *--memory=512m --cpus=1*
- ★ Never store secrets in environment variables or Dockerfiles - use Docker Secrets or a vault